VINMOTION

Giao thức Zenoh

Người báo cáo: Lê Dương Khánh Huy
Ngày: 6/2/2026

1. Tổng quan.....3
2. Cách hoạt động của Zenoh.....3
2.1. Mô hình tổng quan.....3
2.2. Key expression.....4
2.3. Các mô hình giao tiếp trong zenoh.....4
3. Kiến trúc zenoh.....4
3.1. Peer to peer.....5
3.2 Client - Router.....5
4. So sánh Zenoh với các giao thức khác.....6
4.1. Mô hình & kiến trúc hệ thống.....6
4.2. Khả năng định tuyến & quản lý dữ liệu.....6
4.3. Hiệu năng & khả năng chịu lỗi mạng.....7
5. Use case: Ứng dụng truyền thông trong giao tiếp giữa trạm sạc, server và robot.....7
5.1.1. Tổng quan hệ thống.....7
5.1.3. Cơ chế trao đổi dữ liệu bằng Key Expression.....8
5.1.4. Giao tiếp giữa Trạm sạc và Server.....8
5.1.5. Giao tiếp giữa Trạm sạc và Robot.....9
5.1.6. Tổ chức dữ liệu ở tầng ứng dụng.....9
5.1.7. Ví dụ dữ liệu trạm sạc gửi tới Server và Robot.....9
5.1.8. Ví dụ dữ liệu Server gửi tới Trạm sạc.....10
5.1.9. Ví dụ dữ liệu Robot gửi tới Trạm sạc.....10

1. Tổng quan

Zenoh là một giao thức truyền dữ liệu phân tán thế hệ mới, được thiết kế nhằm hợp nhất các mô hình giao tiếp phổ biến như **Publish/Subscribe**, **Query/Reply** và **Data Storage** trong một nền tảng duy nhất.

Zenoh được phát triển bởi **ZettaScale**, hướng tới các hệ thống hiện đại như **IoT, Edge Computing, Robotics và Cloud**, nơi dữ liệu cần được truyền **nhanh, linh hoạt và ổn định**, ngay cả trong điều kiện **mạng không hoàn hảo hoặc không liên tục**.

Khác với các giao thức truyền thống vốn chỉ tập trung vào một mô hình giao tiếp cụ thể, Zenoh tiếp cận theo hướng **data-centric**, trong đó dữ liệu được định danh bằng **key expression** và có thể được **publish, truy vấn hoặc lưu trữ** một cách thống nhất. Điều này giúp đơn giản hóa kiến trúc hệ thống, giảm số lượng giao thức cần triển khai và tăng khả năng mở rộng.

Ngoài ra, Zenoh hỗ trợ nhiều mô hình triển khai như **peer-to-peer, client–router** mà không bắt buộc phải có broker trung tâm, từ đó cải thiện độ trễ, giảm điểm lỗi đơn và phù hợp với các hệ thống phân tán quy mô lớn.

2. Cách hoạt động của Zenoh

2.1. Mô hình tổng quan

Zenoh hoạt động theo mô hình **data-centric**, trong đó **dữ liệu** là trung tâm của hệ thống thay vì các endpoint giao tiếp cố định.

Thay vì gửi dữ liệu trực tiếp từ client A đến client B, Zenoh cho phép các node:

- Xuất bản dữ liệu (publish)
- Đăng ký nhận dữ liệu (subscribe)
- Truy vấn dữ liệu (query)
- Lưu trữ dữ liệu trong mạng (storage)

Tất cả các thao tác này đều dựa trên **key expression** – một không gian tên logic để định danh dữ liệu.

2.2. Key expression

Key expression là chuỗi định danh dữ liệu, ví dụ:

factory/line1/sensor/temperature
robot/pose/x

Đặc điểm:

- Có cấu trúc phân cấp

- Hỗ trợ wildcard (ký hiệu đặc biệt cho phép **khớp nhiều key** (đường dẫn dữ liệu) cùng lúc, thay vì phải chỉ định chính xác từng key cụ thể).
- Không phụ thuộc vào địa chỉ IP hay endpoint cụ thể.

Nhờ có key expression, Zenoh cho phép nhiều publisher xuất bản cùng một loại dữ liệu và nhiều subcriber nhận dữ liệu mà không cần biết nguồn cụ thể hoặc truy vấn theo phạm vi hoặc pattern.

2.3. Các mô hình giao tiếp trong zenoh

1. Publish/Subcribe
    - Publisher gửi dữ liệu gắn với một key và Subscriber đăng ký key (hoặc pattern). Khi có data tới, Subcriber sẽ nhận được ngay.
    - Phù hợp với real-time data, sensor data, …
2. Query/Reply
    - Node gửi query tới một key expression, sau đó các node có dữ liệu sẽ trả lời. Có thể nhận nhiều reply cho 1 query.
    - Phù hợp với việc lấy dữ liệu hiện tại hoặc yêu cầu dữ liệu theo nhu cầu.
3. Storage/cache
    - Dữ liệu có thể được lưu trong mạng zenoh. Trong trường hợp các subcriber gặp sự cố truy cập vào mạng trễ, chúng vẫn có dữ liệu để đọc.

3. Kiến trúc zenoh

Zenoh không áp đặt một kiến trúc mạng cố định mà được thiết kế theo hướng **linh hoạt và phân tán**, cho phép triển khai phù hợp với nhiều quy mô và môi trường khác nhau.

Tùy vào yêu cầu hệ thống, Zenoh có thể hoạt động theo:

- Peer-to-Peer
- Client – Router
- **Kiến trúc lai (Hybrid)**

Điểm chung của mọi mô hình là:

- Không phụ thuộc bắt buộc vào broker trung tâm.
- Định tuyến dữ liệu dựa trên **key expression.**
- Tối ưu cho độ trễ thấp và khả năng mở rộng.

3.1. Peer to peer

Trong kiến trúc **Peer-to-Peer (P2P)**, các node Zenoh kết nối **trực tiếp với nhau**, không cần thông qua router hay broker trung tâm. Mỗi node vừa có thể là **nguồn dữ liệu** (publisher), vừa có thể là **nơi tiêu thụ dữ liệu** (subscriber hoặc querier).

Cách hoạt động:

1. Các node Zenoh thiết lập kết nối trực tiếp (LAN hoặc mạng riêng).
2. Mỗi node:
    - Publish dữ liệu với key expression
    - Subscribe hoặc query dữ liệu theo key
3. Zenoh tự động:
    - Ghép publisher với subscriber phù hợp
    - Chỉ truyền dữ liệu khi có bên quan tâm

Không có thành phần trung tâm điều phối toàn bộ hệ thống.

Ưu điểm: Truyền dữ liệu nhanh, dễ triển khai

Nhược điểm: Khó mở rộng khi số lượng node lớn, discovery phức tạp trong mạng rộng.

3.2 Client - Router

Trong kiến trúc **Client – Router**, các node Zenoh (client) **không giao tiếp trực tiếp với nhau**, mà kết nối thông qua một hoặc nhiều **Zenoh Router**. Router đóng vai trò trung gian trong việc **kết nối, định tuyến và quản lý luồng dữ liệu** giữa các client.

Kiến trúc này được thiết kế để **mở rộng quy mô hệ thống**, đặc biệt phù hợp với các môi trường **WAN, Cloud và hệ thống phân tán lớn**.

Cách hoạt động

1. Các client (sensor, edge device, application) kết nối tới Zenoh Router.
2. Client publish dữ liệu kèm key expression.
3. Router:
    - Nhận dữ liệu
    - Xác định các client đang subscribe phù hợp
    - Chuyển tiếp dữ liệu đến đúng nơi
4. Client nhận dữ liệu mà **không cần biết nguồn phát cụ thể**.

Router có thể được cấu hình để:

- Lưu trữ dữ liệu
- Cache dữ liệu
- Hỗ trợ query từ client mới tham gia

Ưu điểm: Quản lý tập trung, dễ mở rộng khi có nhiều node

Nhược điểm: Phụ thuộc vào Zenoh router, router cần cấu hình và quản lý, độ trễ cao hơn so với peer-to-peer.

4. So sánh Zenoh với các giao thức khác

4.1. Mô hình & kiến trúc hệ thống

Tiêu chí	Zenoh	MQTT	HTTP/REST	TCP/UDP
Mô hình giao tiếp	Pub/Sub + Query + Storage	Pub/Sub	Request/Response	Point to Point
Kiến trúc tổng thể	P2P, Client-Router, Hybrid	Broker-centric	Client–Server	Tự design
Thành phần trung gian bắt buộc	Không	Có	Không	Không
Khả năng tránh Single Point of Failure	Cao	Thấp	Trung bình	Tùy thuộc design

4.2. Khả năng định tuyến & quản lý dữ liệu

Tiêu chí	Zenoh	MQTT	HTTP/REST	TCP/UDP
Discovery	Tự động phân tán	Qua broker	Không	Không
Cơ chế định tuyến	Theo key expression	Topic cố định	Không	Không
Lưu trữ dữ liệu	Có	Không	Không	Không
Query dữ liệu	Có	Không	Không	Không

4.3. Hiệu năng & khả năng chịu lỗi mạng

Tiêu chí	Zenoh	MQTT	HTTP/REST	TCP/UDP
Độ trễ	Rất thấp	Thấp	Cao	Thấp
Overhead giao thức	Theo key expression	Topic cố định	Không	Không
Tối ưu băng thông	Cao	Trung bình	Thấp	Phụ thuộc
Hoạt động trong mạng không ổn định	Tốt	Trung bình	Kém	Kém

5. Use case: Ứng dụng truyền thông trong giao tiếp giữa trạm sạc, server và robot

5.1.1. Tổng quan hệ thống

Hệ thống được thiết kế để phục vụ việc **trao đổi dữ liệu và điều phối hoạt động** giữa các thành phần trong quy trình thay pin tự động, bao gồm ba thành phần chính:

- **Trạm sạc (Charging Station)**
  Cung cấp thông tin về trạng thái hoạt động, tình trạng pin, tiến trình thay pin và các lỗi phát sinh trong quá trình vận hành.
- **Robot**
  Gửi yêu cầu thay pin, cung cấp trạng thái cảm biến hành trình, đồng thời nhận thông tin tiến trình và phản hồi từ trạm sạc.
- **Server**
  Đóng vai trò giám sát và điều phối toàn bộ hệ thống, quản lý nhiều robot và trạm sạc, lưu trữ dữ liệu và đưa ra các quyết định điều hành.

5.1.2. Yêu cầu hệ thống

Hệ thống truyền thông cần đáp ứng các yêu cầu sau:

- Truyền và nhận dữ liệu **thời gian thực với độ trễ thấp**
- Hỗ trợ **nhiều robot và trạm sạc hoạt động đồng thời**
- Hoạt động ổn định trong môi trường **mạng không hoàn hảo hoặc gián đoạn**
- Có khả năng **mở rộng và quản lý tập trung**
- Tách biệt rõ ràng giữa **logic truyền thông** và **logic ứng dụng**

5.1.3. Cơ chế trao đổi dữ liệu bằng Key Expression

Việc trao đổi dữ liệu giữa các thành phần được thực hiện thông qua **key expression của Zenoh** (tương tự topic trong MQTT).

Mỗi key expression được thiết kế để phục vụ **một chiều giao tiếp duy nhất**, nghĩa là:

- Một key chỉ dùng để **gửi (publish)**
- Một key chỉ dùng để **nhận (subscribe)**

Key expression được xây dựng **linh hoạt theo ID hoặc tên của robot và trạm sạc**, giúp hệ thống dễ dàng mở rộng khi số lượng node tăng lên.

5.1.4. Giao tiếp giữa Trạm sạc và Server

Đối với mỗi trạm sạc, hệ thống sử dụng hai key expression:

- **Trạm sạc** → **Server** (gửi trạng thái)
  `<charger_name>/charger/status`
- **Server** → **Trạm sạc** (gửi yêu cầu điều khiển)

  `<charger_name>/charger/server_request`

Cách tổ chức này cho phép server:

- Theo dõi trạng thái của nhiều trạm cùng lúc
- Điều khiển từng trạm một cách độc lập
- Dễ dàng subscribe nhiều trạm bằng wildcard

5.1.5. Giao tiếp giữa Trạm sạc và Robot

Giao tiếp giữa trạm sạc và robot cũng được tổ chức theo cặp key expression riêng biệt:

- **Trạm sạc** → **Robot** (gửi tiến trình thay pin)

  `<robot_name>/charger/progress`

- **Robot** → **Trạm sạc** (gửi yêu cầu và trạng thái cảm biến)
  `<charger_name>/charger/robot_request`

Nhờ việc gắn ID robot và trạm sạc trực tiếp trong key expression, hệ thống không cần duy trì bảng ánh xạ phức tạp và có thể mở rộng linh hoạt.

5.1.6. Tổ chức dữ liệu ở tầng ứng dụng

Ở tầng ứng dụng, dữ liệu được tổ chức dưới dạng **JSON** nhằm đảm bảo:

- Dễ đọc và dễ debug
- Linh hoạt khi mở rộng cấu trúc dữ liệu
- Phù hợp cho cả server, robot và trạm sạc

Dữ liệu JSON sẽ được:

- Serialize thành chuỗi byte
- Truyền qua các API của Zenoh
- Deserialize ở phía nhận để xử lý logic ứng dụng

5.1.7. Ví dụ dữ liệu trạm sạc gửi tới Server và Robot

```
{
  "charger_name": "name",
  "charger_MAC": "MAC address",
  "battery": {
    "1": 0.1,
    "2": 0.1,
    "3": 0.1,
    "4": 0.1,
    "5": 0.1
  },
  "status": "IDLE",
  "charging": {
    "progress": "DOING_ABOVE",
    "error": {
      "id": "",
      "detail": ""
    },
    "estimate_time": 10
  }
}
```

Trong đó:

- `status`: trạng thái tổng thể của trạm (IDLE, WAITING, BUSY, ERROR)
- `progress`: tiến trình thay pin
- `battery`: thông tin pin hiện có trong trạm
- `error`: thông tin lỗi nếu xảy ra

5.1.8. Ví dụ dữ liệu Server gửi tới Trạm sạc

```
{
  "robot_name": "robot_name",
  "request": "WAITING"
}
```

Server sử dụng dữ liệu này để:

- Thông báo robot sắp tới trạm

- Điều khiển trạng thái chờ hoặc hủy thao tác (`CANCEL`) khi cần thiết.

5.1.9. Ví dụ dữ liệu Robot gửi tới Trạm sạc

```
{
  "robot_name": "robot_name",
  "mac_address": "mac_address",
  "request": true,
  "upper_sensor_on": true,
  "lower_sensor_on": false
}
```

Dữ liệu này cho phép trạm sạc:

- Xác nhận yêu cầu thay pin từ robot
- Theo dõi trạng thái cảm biến hành trình để xác định tiến trình lắp pin